



POO - Unid IV

Exercício 1:

Considere o trecho de código abaixo como um excerto (parte de um método maior). Assuma que o arquivo já tem as diretivas `using System;` e `using System.IO;`, que `caminhoArquivo` é uma variável do tipo `string`, já inicializada e no escopo desse `try`.

```
try
{
    ProcessarArquivo(caminhoArquivo);
}
catch (FileNotFoundException ex) when (caminhoArquivo.EndsWith(".txt"))
{
    Console.WriteLine("Arquivo de texto não encontrado. Crie o arquivo antes de executar.");
}
catch (FileNotFoundException ex)
{
    Console.WriteLine($"Arquivo não encontrado: {ex.FileName}");
}
```

Nessas condições, suponha que `ProcessarArquivo(caminhoArquivo)` lance `FileNotFoundException`. Qual alternativa descreve corretamente o critério de seleção do bloco `catch` e a consequência quando a condição do `when` falha?

A)

O primeiro `catch` é sempre escolhido por aparecer antes; a condição após `when` afeta apenas o corpo executado, mas não a captura.

B)

Se o `when` for falso, a exceção é considerada capturada mesmo assim e é reencaminhada diretamente para fora do `try`, ignorando qualquer `catch` seguinte.

C)

Se o `when` for verdadeiro, o primeiro `catch` é executado; se o `when` for falso, a exceção não é capturada por aquele `catch` e pode ser capturada por um `catch`

compatível posterior.

D)

O when só é avaliado depois de o primeiro catch ter sido executado; se falhar, ocorre uma nova exceção indicando "filtro inválido".

E)

O when muda o tipo efetivo da exceção para um subtipo filtrado; por isso, o segundo catch (FileNotFoundException ex) se torna inalcançável.

Exercício 2:

Em uma aplicação que recebe um denominador a partir de uma entrada externa, qual decisão de design está mais consistente com a distinção entre (i) situações previsíveis do domínio e (ii) violações de contrato que justificam exceções em sentido forte?

A)

Tratar o caso de denominador igual a zero exclusivamente por catch (DivideByZeroException), pois a exceção é a forma idiomática de lidar com entradas inválidas em C#.

B)

Validar explicitamente o denominador antes da divisão e encaminhar o caso de zero por lógica de controle e retorno adequado; deixar DivideByZeroException como sintoma de uma pré-condição que não foi garantida em algum ponto.

C)

Remover toda validação de entrada e, quando ocorrer divisão por zero, encapsular a exceção em Exception sem InnerException, para simplificar logs e padronizar mensagens.

D)

Converter toda divisão por zero em um valor sentinela (por exemplo, double.NaN) e seguir o fluxo normalmente, pois isso elimina a necessidade tanto de validação

quanto de tratamento de erros.

E)

Mover a validação para o compilador por meio de anotações de tipo, pois a linguagem consegue detectar estaticamente que entradas externas nunca serão zero.

Exercício 3:

Sobre testes automatizados, regressões, integração contínua e a organização conceitual de um teste no ecossistema .NET moderno, qual alternativa é a mais compatível com as práticas usuais?

A)

Um teste de regressão deve ficar em um projeto separado e só deve ser executado antes de uma release, para evitar que a suíte "diária" cresça.

B)

Um teste pode ser organizado como preparação, ação, verificação e, quando necessário, limpeza; ao executar continuamente a suíte em pipelines de integração contínua, mudanças recentes passam por reexecução recorrente que ajuda a detectar regressões.

C)

Em integração contínua, é preferível executar apenas testes de integração, porque testes unitários tendem a aumentar falsos positivos ao acoplar a suíte a detalhes internos.

D)

Após corrigir um defeito, não se deve criar um teste específico que o reproduza; a validação deve ficar restrita a testes exploratórios manuais para não "engessar" a suíte.

E)

Como erros de lógica, muitas vezes, não geram exceção, a prática recomendada é envolver cada função em try/catch e registrar apenas Message, evitando StackTrace em logs.

Exercício 4:

Em um projeto de testes em C# que compila sem erros, existem métodos marcados com [Fact] e chamadas a Assert.*. O projeto já referencia xunit e Microsoft.NET.Test.Sdk, mas o Gerenciador de Testes do Visual Studio lista zero testes. Qual alteração é a mais diretamente relacionada a restaurar a descoberta e a execução no Gerenciador de Testes?

A)

A adição do pacote xunit.runner.visualstudio, mantendo as demais referências já existentes.

B)

A adição de uma referência de projeto ao assembly de produção, mesmo que os testes não usem tipos desse assembly.

C)

A substituição do atributo [Fact] pelo atributo [Theory] em todos os métodos, mesmo sem dados de entrada.

D)

A remoção do pacote Microsoft.NET.Test.Sdk, pois ele conflita com o mecanismo de descoberta do Visual Studio.

E)

A migração para um modelo com métodos estáticos de [SetUp] e [TearDown], para que o Visual Studio reconheça o ciclo de vida dos testes.

Exercício 5:

Considere uma classe de testes em xUnit que implementa `IDisposable`, usa um construtor para inicialização e contém um método com `[Fact]` e um método com `[Theory]` acompanhado de três `[InlineData]`. Ao executar "Executar Todos os Testes" no Gerenciador de Testes do Visual Studio, qual alternativa descreve corretamente (i) o total de testes reportados, (ii) quantas instâncias da classe de teste são criadas e (iii) quantas vezes `Dispose()` é chamado?

A)

São reportados 2 testes; é criada 1 instância; e `Dispose()` é chamado 1 vez, ao final da execução de todos os testes.

B)

São reportados 2 testes; são criadas 2 instâncias; e `Dispose()` é chamado 2 vezes, uma após cada método.

C)

São reportados 4 testes; são criadas 4 instâncias; e `Dispose()` é chamado 4 vezes, uma após cada caso executado.

D)

São reportados 3 testes; são criadas 3 instâncias; e `Dispose()` é chamado 1 vez, ao final do último caso do `[Theory]`.

E)

São reportados 5 testes; são criadas 2 instâncias; e `Dispose()` é chamado 5 vezes, pois o `[Theory]` chama `Dispose()` a cada `InlineData`, mas o `[Fact]` não.

Exercício 6:

Em uma implementação em C#, existe uma classe `Perfil` com os métodos `Curtir(Perfil outro)`, `TemMatchCom(Perfil outro)` e `EhCompatívelCom(Perfil outro)`, com as seguintes regras:

- `Curtir` retorna `true` somente quando a curtida é mútua; nesse caso, registra o match nos dois perfis. Quando ainda não é mútua, registra a curtida em uma lista interna (para permitir reciprocidade futura) e retorna `false`.
- `TemMatchCom` retorna `true` se o outro perfil estiver na lista interna de matches.
- `EhCompatívelCom` exige pelo menos um interesse em comum via interseção

sensível a maiúsculas/minúsculas e exige a mesma localização por comparação que ignora maiúsculas/minúsculas.

Considere dois perfis inicialmente sem curtidas e sem matches, com:

- a.Localizacao = "São Paulo" e b.Localizacao = "são paulo"
- a.Interesses = ["Música"] e b.Interesses = ["música"]

Considere a sequência:

I) o resultado de a.EhCompatívelCom(b)

II) o retorno da primeira chamada a.Curtir(b)

III) o retorno da primeira chamada b.Curtir(a) após II

IV) o resultado de a.TemMatchCom(b) após II e III

Qual alternativa apresenta corretamente a tupla (I, II, III, IV)?

A)

(true, true, false, false)

B)

(true, false, true, true)

C)

(false, false, true, true)

D)

(false, true, false, true)

E)

(false, true, true, false)

Exercício 7:

Um time decide representar um conceito de domínio em C#. A intenção é reduzir pressão sobre o heap gerenciado e manter um raciocínio claro sobre “quem possui” determinado estado, sem perder propriedades essenciais do tipo. Qual alternativa descreve, com maior fidelidade, o conjunto de consequências atribuídas a struct e a class no modelo apresentado?

A)

Uma struct é sempre alocada na pilha; logo, ela nunca participa de compactação do GC e nunca pode aparecer no heap, ainda que seja colocada em um array.

B)

Uma class pode ser alocada na pilha quando o objeto é pequeno, pois o CLR escolhe o local de alocação conforme o tamanho e a frequência de acesso.

C)

Uma struct tem semântica de valor e pode existir como variável local ou embutida em outros objetos; ela reside no heap quando faz parte de um objeto no heap (por exemplo, como elemento de um array ou como campo de uma class) e pode sofrer boxing ao ser convertida para object ou para uma interface. Além disso, struct é implicitamente selada e não suporta herança de implementação; ela pode declarar construtor sem parâmetro (C# 10+). Já class é tipo por referência, normalmente alocado no heap e coletado pelo GC.

D)

Uma struct suporta herança e por isso é indicada para objetos complexos; uma class é indicada para dados pequenos, pois cópias de referência custam mais do que cópias de valor.

E)

Uma struct impede completamente o boxing em coleções genéricas, pois o CLR desvia automaticamente para uma representação por referência quando necessário.

Exercício 8:

Em uma aplicação com interface gráfica e também em um servidor que atende a requisições, surgem travamentos difíceis de reproduzir e quedas de responsividade em trechos dominados por espera de rede e disco. Com base no modelo descrito para concorrência e assincronia no ecossistema .NET, qual alternativa explica melhor por que Task com async/await e cancelamento cooperativo se encaixam nesse cenário, e também por que isso não elimina a necessidade de entender fundamentos de execução?

A)

Porque `async/await` cria automaticamente uma thread dedicada para cada operação de I/O, garantindo que nunca haja alternância de núcleo e eliminando condições de corrida.

B)

Porque o foco em Task desloca a atenção da mecânica de criação/sincronização de threads para unidades de trabalho compostas; em operações I/O-bound, isso evita esperas bloqueantes que congelariam a interface ou reduziriam a capacidade do servidor, e o cancelamento via `CancellationToken` permite interromper de modo cooperativo operações longas sem recorrer a mecanismos abruptos e sem deixar recursos pendurados; ainda assim, permanece a responsabilidade de compreender o que o runtime faz com threads, tarefas e agendamento.

C)

Porque Task substitui completamente o escalonador do sistema operacional; por isso o comportamento fica determinístico e o raciocínio sobre interleavings deixa de ser relevante.

D)

Porque o cancelamento em .NET funciona por preempção do runtime: ele aborta a thread imediatamente e, por construção, não há risco de recursos permanecerem abertos.

E)

Porque o uso de Task impede pausas do garbage collector, já que o GC não suspende threads quando existe código assíncrono no processo.

Exercício 9:

Em um método assíncrono em C# que retorna `Task<int>` e contém um `await` aplicado a uma Task ainda não concluída, qual sequência de eventos descreve, com maior fidelidade, o que ocorre entre a chamada do método e a obtenção do resultado pelo chamador?

A)

O compilador preserva o corpo do método como está, e o runtime executa o `await` como uma chamada bloqueante; o método só retorna ao chamador depois que a

operação termina, garantindo que a thread original permaneça ativa até o fim.

B)

O compilador transforma o método em uma máquina de estados; ao encontrar um await de uma Task incompleta, ele registra uma continuação para retomar o fluxo após a conclusão e devolve o controle ao ambiente de chamada; quando a Task termina com sucesso, a continuação retoma e o resultado passa a estar disponível no ponto do await.

C)

O runtime converte automaticamente o método em um novo processo leve para isolar o espaço de endereçamento; o await apenas sincroniza o encerramento do processo e a devolução do valor ao chamador.

D)

O await força a criação de uma nova thread do sistema operacional para a continuação; essa continuação sempre roda em uma thread recém-criada, pois a reutilização de threads impediria a retomada correta do contador de programa.

E)

A Task aguardada substitui o escalonamento do sistema operacional; durante a espera, não há threads disponíveis para outras tarefas no mesmo processo, pois a thread chamadora permanece reservada até a conclusão.

Exercício 10:

Em uma aplicação com interface gráfica, um manipulador de evento chama `httpClient.GetStringAsync(uri)`. Considerando a distinção entre operações I/O-bound e CPU-bound e a forma como Task representa uma operação em andamento, qual afirmação descreve corretamente o efeito desse padrão sobre a responsividade da aplicação?

A)

A chamada assíncrona mantém uma thread do thread pool bloqueada até a chegada dos bytes da rede, mas isso costuma ser aceitável porque o thread pool cria uma thread adicional para cada requisição pendente.

B)

A chamada assíncrona retorna imediatamente um `Task<string>`; enquanto a resposta não chega, a thread que executava o manipulador não fica bloqueada e pode voltar a processar outras mensagens; quando a operação termina, a continuação do método é reagendada e o texto é entregue ao ponto do `await`.

C)

A chamada assíncrona é equivalente a executar `Task.Run(() => httpClient.GetString(uri))`; o ganho de responsividade decorre de deslocar a espera de rede para uma computação paralela em CPU.

D)

A chamada assíncrona evita a espera por rede eliminando a latência do servidor; por isso, ela sempre conclui em tempo menor do que a forma síncrona, independentemente do tráfego e do tempo de resposta.

E)

A chamada assíncrona funciona apenas porque o `await` obriga o runtime a manter o manipulador em execução contínua, impedindo que o loop de mensagens assuma o controle até a conclusão da requisição.